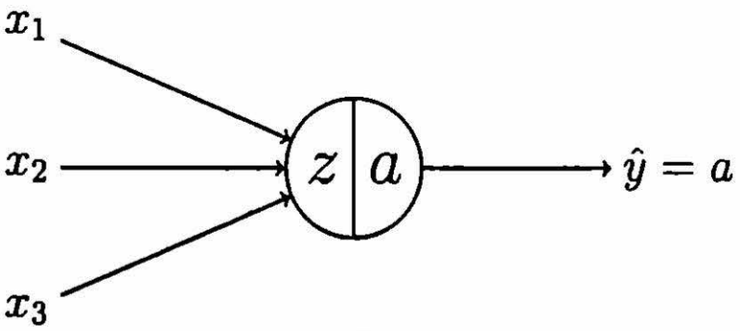
Chương 3

Mạng đa tầng và học sâu

3.1 Hồi quy logistic dưới góc nhìn mạng nơ-ron

Như đã trình bày trong chương trước, *hồi quy logistic* là một phương pháp *phân loại nhị phân* (binary classification method), được xây dựng bởi một hàm có khả năng nhận một giá trị bất kỳ và trả kết quả ra một con số nằm giữa 0 và 1 (hàm *sigmoid*). Điều này cũng tương tự như một *mạng nơ-ron* (neural network) một lớp.



$$z = w^T x + b$$

$$a = \sigma(z) = \frac{1}{1+e^z}$$

Hình 3.1: Một mạng nơ-ron đơn giản

Hình 3.1 minh họa cho việc dùng hồi quy logistic như một mạng nơ-ron đơn giản. Trong đó, vector x chứa các *thuộc tính* (feature) của dữ kiện đầu vào (input), vector w sẽ là trọng số các thuộc tính và b là giá trị bias. Sau khi trải qua các bước tính toán, kết quả thu được $\hat{y}$ sẽ là xác suất mà *nhãn* (label) y nhận giá trị bằng 1 với x và w đã biết như sau:

$$P(y = 1|w, x) = \hat{y}$$

Mặc dù kết quả của hàm σ luôn là một giá trị từ 0 đến 1 nhưng giá trị của nhãn thì luôn là 0 hoặc 1. Sự sai biệt này dẫn đến khái niệm *hàm mất mát* (loss function). Có rất nhiều hàm có thể được dùng để làm hàm mất mát, tuy nhiên trong chương này ta sẽ chỉ dùng *hàm mất mát cross-entropy* như đã trình bày trong Chương 2, được nhắc lại như sau.

Công thức hàm Cross-entropy:

$$L(\hat{y}, y) = -[y \log \hat{y} + (1 - y) \log (1 - \hat{y})]$$

Công thức hàm chi phí:

$$J = -\frac{1}{m} \sum_{i=1}^{m} L(\hat{y}^{(i)}, y^{(i)})$$

3.2 Mạng nơ-ron đa tầng và mạng học sâu

3.2.1 Nhắc lại mạng nơ-ron đa tầng

Như chương trước đã đề cập, mạng nơ-ron đa tầng (multilayer perceptron) thực chất chỉ gồm nhiều tầng mà trong mỗi tầng các perceptron đặt chồng lên nhau, các tầng được nối với nhau theo cơ chế *fully-connected*. Cách thức hoạt động của những tầng mạng cũng rất đơn giản bằng các phép nhân ma trận. Bằng cách này, chúng ta có thể giải quyết rất nhiều bài toán mà một perceptron không thể xử lý được. Tuy nhiên, bên cạnh

hàm kích hoạt sigmoid như đã nêu ở chương trước thì các nghiên cứu về mạng nơ-ron đa tầng đưa ra một số dạng hàm kích hoạt khác.

3.2.2 Hàm kích hoạt của mạng nơ-ron đa tầng

Quá trình xây dựng hàm kích hoạt sẽ được tiến hành dựa trên những luận điểm như sau:

- Hàm kích hoạt này có thể được thiết kế như một hàm phi tuyến để giải quyết những bài toán phi tuyến.
- Khi đi qua nhiều lớp thì các trọng số có thể suy biến, tức trở nên quá lớn hoặc quá nhỏ, sẽ ảnh hưởng đến độ chính xác của bài toán. Để giải quyết, ta có thể xây dựng một hàm kích hoạt sao cho những giá trị đầu ra sẽ nằm trong một khoảng nào đó có thể kiểm soát được hoặc có ý nghĩa trong xác suất thống kê như (-1,1) hoặc (0,1).
- Khi cần thể hiện độ tin cậy của kết quả, hàm kích hoạt sẽ được xây dựng sao cho giá trị đầu ra nằm trong khoảng (0,1) để thể hiện cho xác suất.

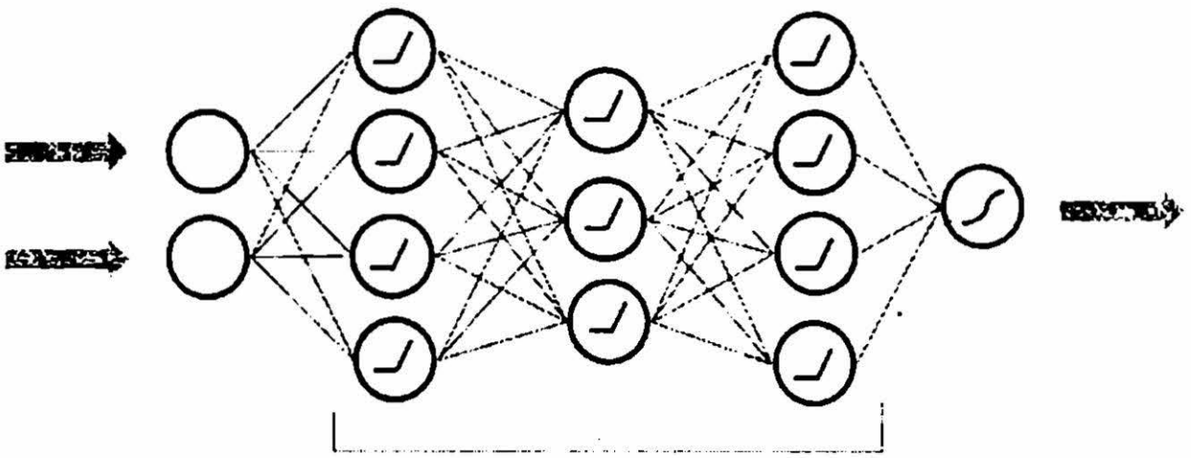
Như vậy, chúng ta sẽ thiết kế hàm kích hoạt sao cho đó là hàm phi tuyến (nhưng phải khả đạo hàm) và đầu ra nên có giá trị (-1,1) hoặc (0,1). Sau một thời gian phát triển, các mạng học sâu đã ra đời và sử dụng những hàm kích hoạt này. Chi tiết của các hàm kích hoạt này sẽ được trình bày ở các phần sau.

3.2.3 Mạng học sâu

Từ khái niệm mạng đa tầng thì chúng ta dẫn đến khái niệm mạng *học sâu* (deep learning - DL) (Schmidhuber, 2015). Nói một cách đơn giản thì mạng học sâu có thể được xem như một mạng đa tầng với số tầng là khá nhiều, trong đó số lượng perceptron ở mỗi tầng có thể khác nhau và mỗi tầng có thể sử dụng một hàm kích hoạt khác nhau. Điều này khác

với mạng đa tầng truyền thống ban đầu chỉ có một hàm kích hoạt dùng chung cho toàn mạng.

Hình 3.2 trình bày tham khảo một kiến trúc một mạng học sâu với 5 tầng có số lượng perceptron mỗi tầng khác nhau, và hàm kích hoạt được dùng cho các tầng cũng khác nhau (ReLU và sigmoid).



Hình 3.2: Kiến trúc mạng học sâu với 3 tầng ẩn

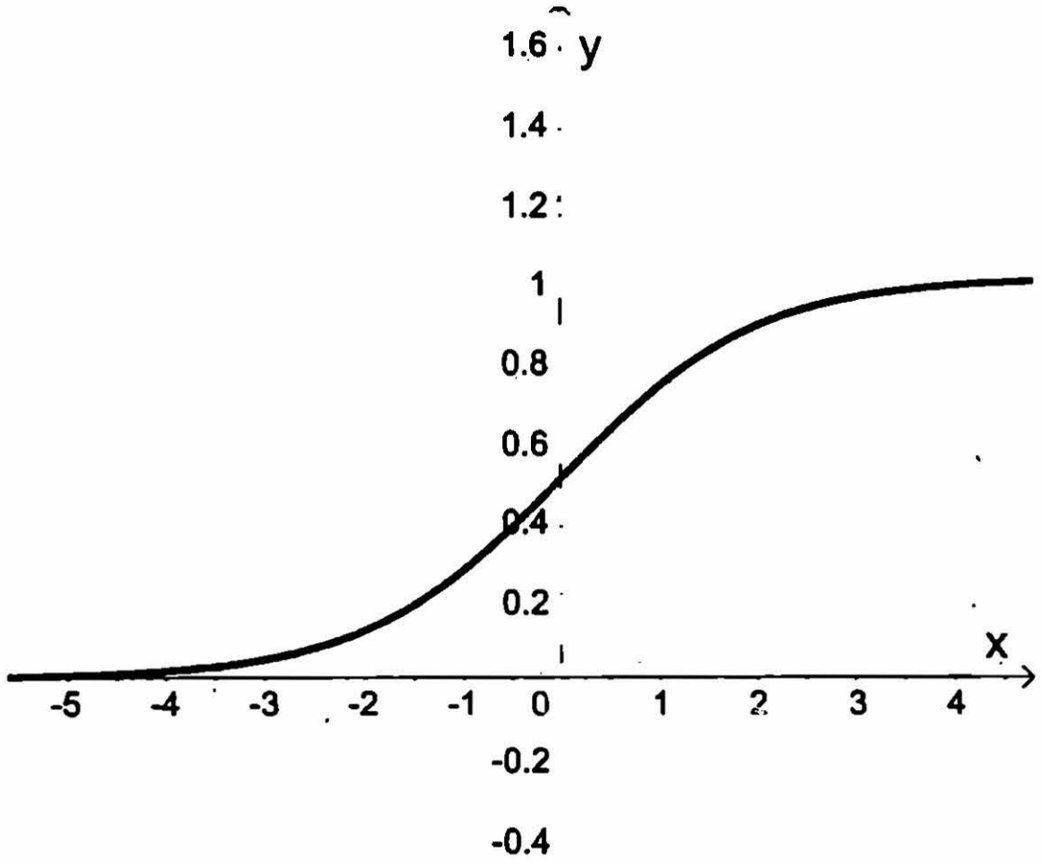
3.3 Các hàm kích hoạt

3.3.1 Hàm sigmoid

Công thức:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{3.1}$$

Hàm sigmoid nhận vào một giá trị thực x và trả về một giá trị trong khoảng $(0, 1)$. Nếu x là một số thực âm rất nhỏ thì kết quả của hàm sigmoid sẽ tiệm cận 0, và ngược lại nếu x là một số dương rất lớn thì kết quả sẽ tiệm cận 1. Hình 3.3 là đồ thị biểu diễn cho hàm sigmoid.



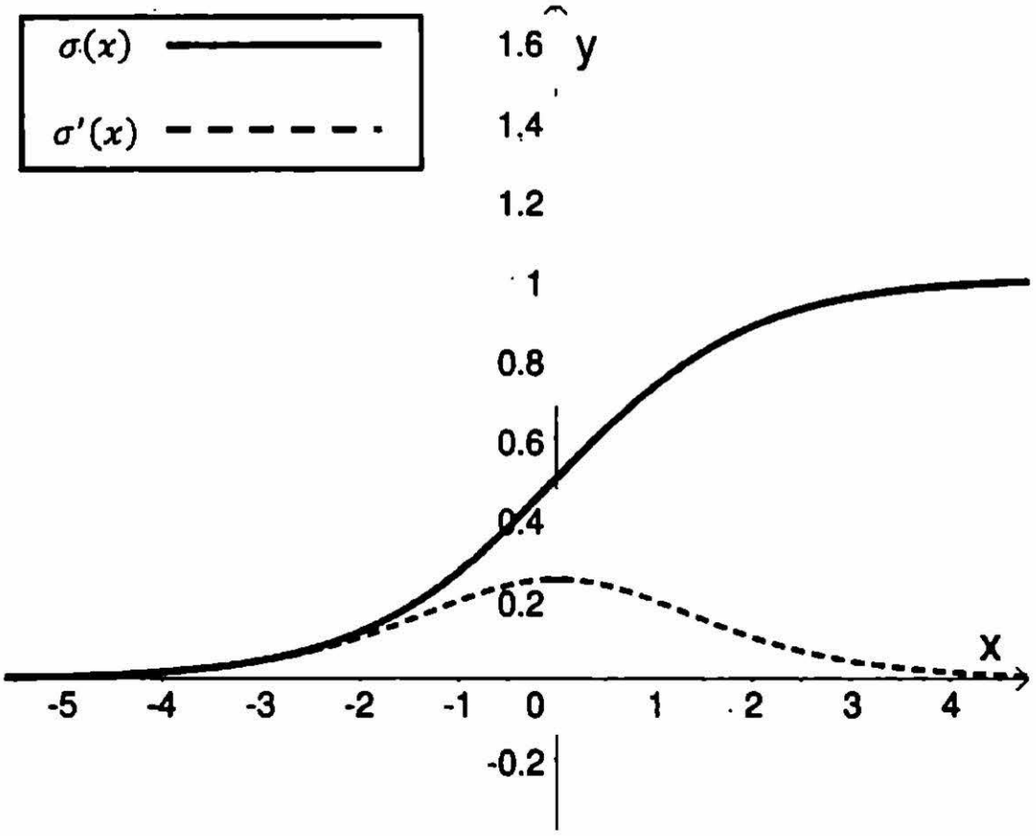
Hình 3.3: Đồ thị của hàm sigmoid

Như đã thấy ở phần 3.1, khi sử dụng hàm sigmoid như hàm kích hoạt, việc tính toán vô cùng thuận lợi do kết quả đạo hàm của sigmoid rất "đẹp". Tuy nhiên, điều này không thể che lấp những khuyết điểm sau của hàm kích hoạt này:

1. **Hiện tượng *triệt tiêu gradient* (vanishing gradient)**

   Trên Hình 3.4, đường phía trên thể hiện cho giá trị của hàm sigmoid và đường phía dưới thể hiện cho giá trị của đạo hàm. Có thể nhận thấy, với những giá trị x rất lớn hoặc rất nhỏ, kết quả đạo hàm của hàm sigmoid rất gần với 0. Điều này gây ra sự triệt tiêu gradient và hạn chế khả năng học của mạng. Cụ thể, nếu mạng được khởi động bằng những trọng số quá lớn hoặc quá nhỏ, giá trị đầu vào của hàm sigmoid bị bão hòa, giá trị của đạo hàm sẽ là một giá trị gần 0 và gradient sẽ bị triệt tiêu. Nếu mạng được khởi động bằng những trọng số "đẹp" (không quá lớn, không quá nhỏ), giá trị của đạo hàm cũng sẽ là một giá trị trong khoảng (0,0.25). Khi đi qua

một mạng nhiều tầng, đạo hàm của các trọng số sẽ nhỏ dần và gradient vẫn sẽ bị triệt tiêu.



Hình 3.4: Đồ thị biểu diễn giá trị và đạo hàm của hàm sigmoid

2. **Hàm sigmoid không có tính chất *zero-centered***

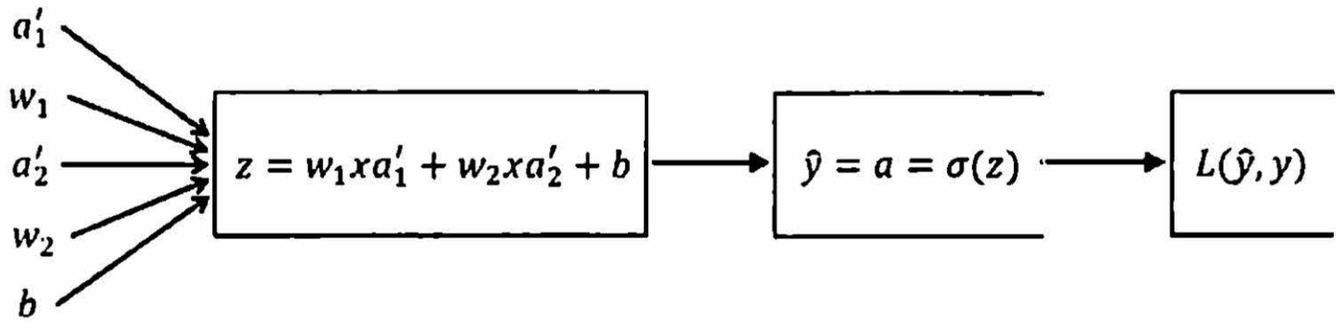
Ở ví dụ của mạng nơ-ron như Hình 3.5, đạo hàm riêng từng phần của hàm mất mát theo hai trọng số w_1 và w_2 sẽ được tính như sau:

$$\nabla w_1 = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_1} = \frac{\partial L}{\partial z} a_1^{'} \tag{3.2}$$

$$\nabla w_2 = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w_2} = \frac{\partial L}{\partial z} a_2^{'} \tag{3.3}$$

Vì $a_1^{'}$ và $a_1^{'}$ là kết quả của một hàm sigmoid trước đó, do đó luôn nhận giá trị dương và dấu của các gradient sẽ phụ thuộc vào $\frac{\partial L}{\partial z}$. Điều này có nghĩa là các gradient sẽ luôn cùng dương hoặc luôn

cùng âm. Việc cập nhật trọng số sẽ chỉ xảy ra về một phía, hạn chế sự linh hoạt của mạng và gây khó khăn cho việc hội tụ.

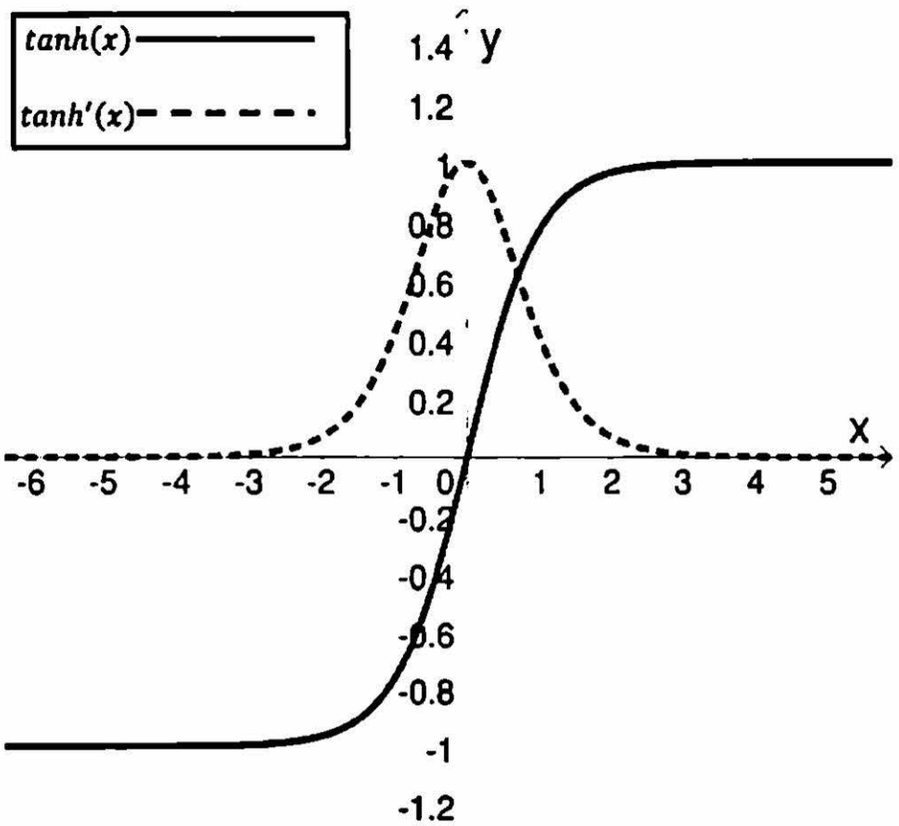


Hình 3.5: Hoạt động của một tầng ẩn trong mạng MLP dùng hàm sigmoid

3.3.2 Hàm tanh

Công thức:

$$tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \tag{3.4}$$



Hình 3.6: Đồ thị của hàm tanh và đồ thị đạo hàm của hàm tanh

Hàm tanh nhận vào một số thực và trả về một giá trị trong khoảng (-1,1). Trên Hình 3.6, đường nét liền thể hiện giá trị của hàm tanh và đường nét đứt thể hiện cho giá trị đạo hàm. Dễ dàng nhận thấy rằng hàm tanh cũng gặp phải vấn đề triệt tiêu gradient như hàm sigmoid. Tuy nhiên, so với sigmoid, hàm tanh có tính chất zero-centered.

Đặc biệt. hàm tanh cũng có thể được biểu diễn bằng hàm sigmoid như biểu thức sau:

$$tanh(x) = 2\sigma(2x) - 1 \tag{3.5}$$

3.3.3 Hàm ReLU

Công thức:

$$f(x) = max(0, x) \tag{3.6}$$

Quan sát công thức (3.6) và Hình 3.7, dễ dàng nhận ra cách hoạt động của hàm ReLU là lọc ra các giá trị đầu vào nhỏ hơn 0. Đạo hàm của ReLU sẽ như sau:

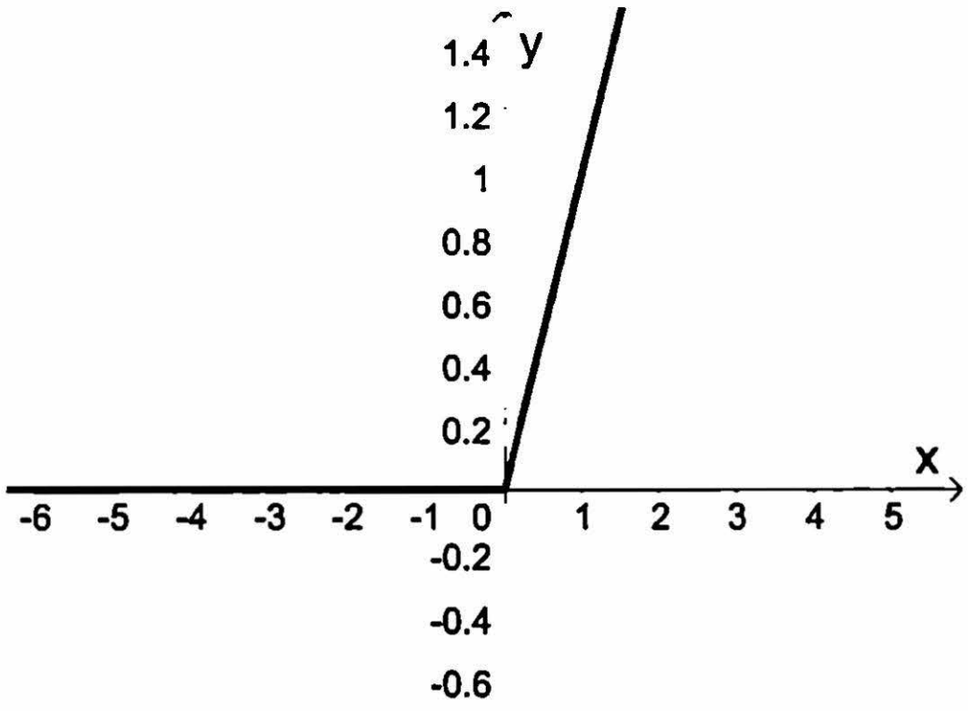
$$f'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{else} \end{cases} \tag{3.7}$$

Như vậy, so với sigmoid và tanh, hàm ReLU sẽ không xuất hiện vấn đề triệt tiêu gradient. Tốc độ tính toán của hàm ReLU cũng sẽ nhanh hơn so với hai hàm trước đó. Tuy nhiên ReLU cũng tồn đọng một nhược điểm, với x có giá trị nhỏ hơn 0, qua hàm ReLU sẽ thu được kết quả bằng 0. Nếu giá trị của node bị chuyển thành 0 thì sẽ không có ý nghĩa ở lớp tiếp theo và các hệ số tương ứng từ node đấy cũng không được cập nhật với gradient. Hiện tượng này gọi là *Dying ReLU* (Lu Lu and Karniadakis, 2019).

3.3.4 Hàm leaky ReLU

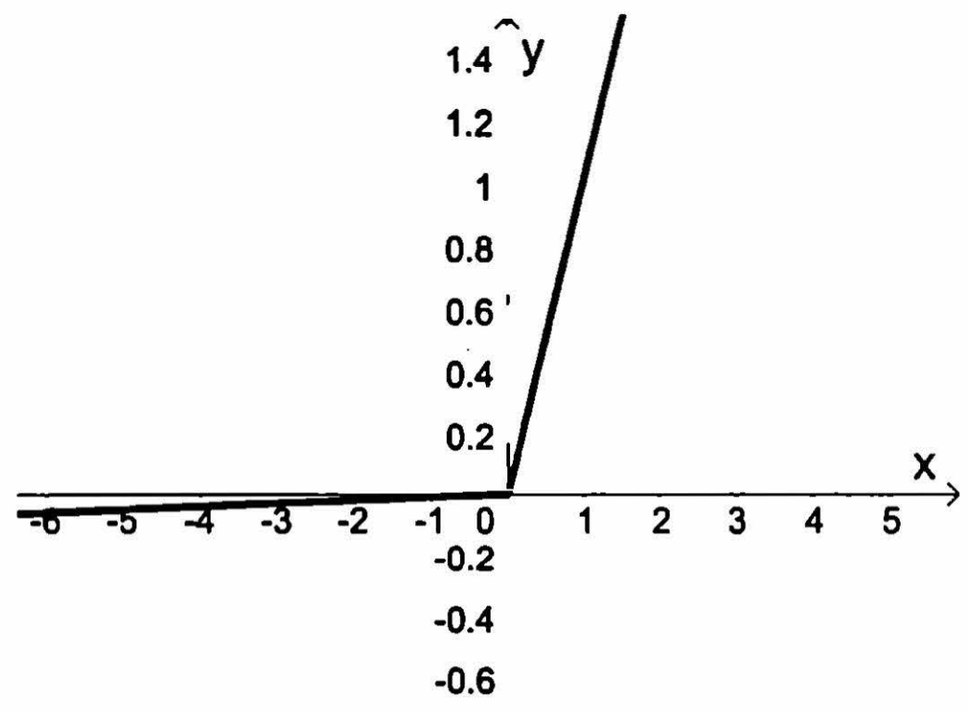
Công thức:

$$f(x) = max(0.01x, x) \tag{3.8}$$



Hình 3.7: Đồ thị biểu diễn hàm ReLU

Leaky ReLU là một biến thể của ReLU để cố gắng xử lý hiện tượng Dying ReLU. Thay vì luôn trả về giá trị bằng 0 cho các giá trị âm, leaky ReLU tạo một đường xiên có độ dốc nhỏ như Hình 3.8. Leaky ReLU cũng có một biến thể khác là PReLU: $f(x) = max(\alpha x, x)$ với α sẽ được chọn trong quá trình học.



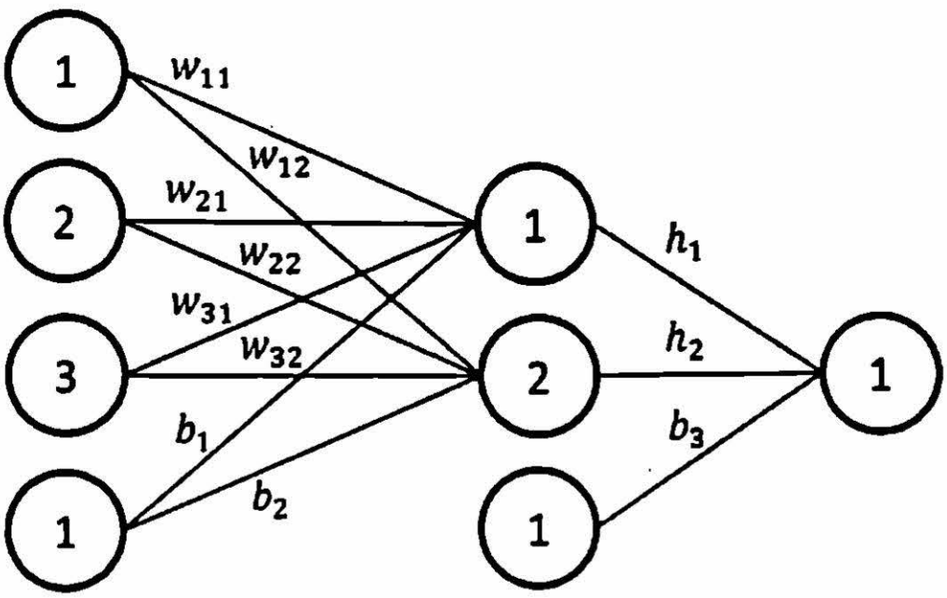
Hình 3.8: Đồ thị biểu diễn hàm Leaky ReLU

3.4 Biểu diễn mạng nơ-ron bằng vector và ma trận

3.4.1 Biểu diễn mạng nơ-ron

Xem xét mạng nơ-ron gồm 3 layer như Hình 3.9. Khi đó, với f là một hàm kích hoạt, ta có:

$$\begin{aligned} h_1 &= f(in_1 \times w_{11} + in_2 \times w_{21} + in_3 \times w_{31} + bias_1) \\ h_2 &= f(in_1 \times w_{12} + in_2 \times w_{22} + in_3 \times w_{32} + bias_2) \\ out &= f(h_1 \times w_{11} + h_2 \times w_{21} + bias_3) \end{aligned} \tag{3.9}$$

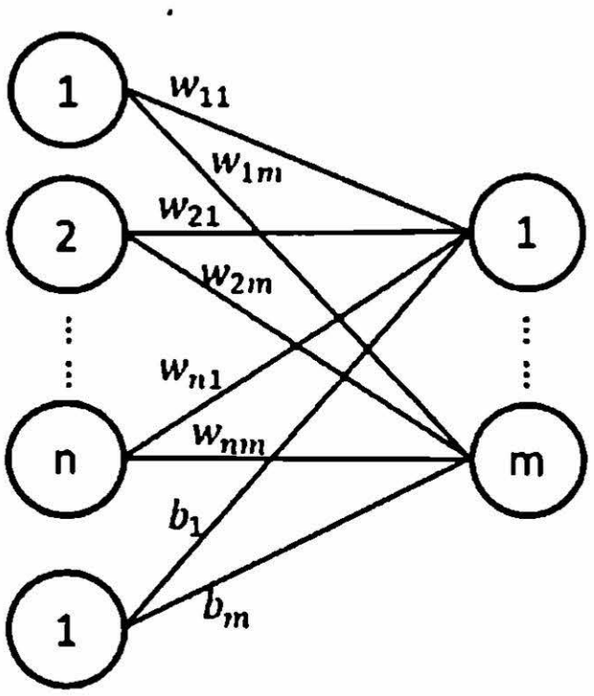


Hình 3.9: Kiến trúc mạng nơ-ron 3 tầng

Với cách xử lý dữ liệu truyền thống, ta có thể lưu các giá trị w_i vào một mảng hay một vector và xử lý tuần tự. Tuy nhiên, cần để ý rằng, quá trình tính toán ở mỗi perceptron sẽ được tính bằng cách lấy giá trị đầu ra của tầng trước đó hoặc là giá trị đầu vào nhân với trọng số tương ứng của nó, quá trình này sẽ được biểu diễn bằng kí hiệu toán học như sau :

$$h_i = f\left(\sum_{j=1}^{num_input} in_j \times w_{ji} + bias_i\right) \tag{3.10}$$

Trong đó, *num_input* chính là số lượng dữ liệu đầu vào được dùng để tính toán trong tầng hiện tại. Công thức này thực chất tương đương với việc nhân 2 ma trận với nhau. Thật vậy, mỗi đầu ra của một perceptron là một giá trị vô hướng (scalar) và do mối quan hệ giữa 2 tầng liên tiếp là như nhau nên, không mất tính tổng quát, chúng ta chỉ xét riêng trường hợp với 2 tầng, như Hình 3.10). Nếu biểu diễn dữ liệu đầu vào thành một vector hay nói cách khác là một ma trận X_{nx1} thì sẽ xây dựng được ma trận trọng số W tương ứng với n hàng, m cột với w_{ij} là trọng số từ input i nối với một output j.



Hình 3.10: Kiến trúc của 2 tầng liên tiếp nhau trong một mạng nơ-ron đa tầng

Dựa theo công thức (3.10) có thể thấy rằng chỉ cần lấy cột thứ i ra làm vector để nhân với vector trong ma trận cột X và áp dụng hàm kích hoạt thì sẽ có kết quả của perceptron thứ i. Nếu lúc này xét h_{mx1} là ma trận của các perceptron đầu ra thì sẽ có công thức như sau:

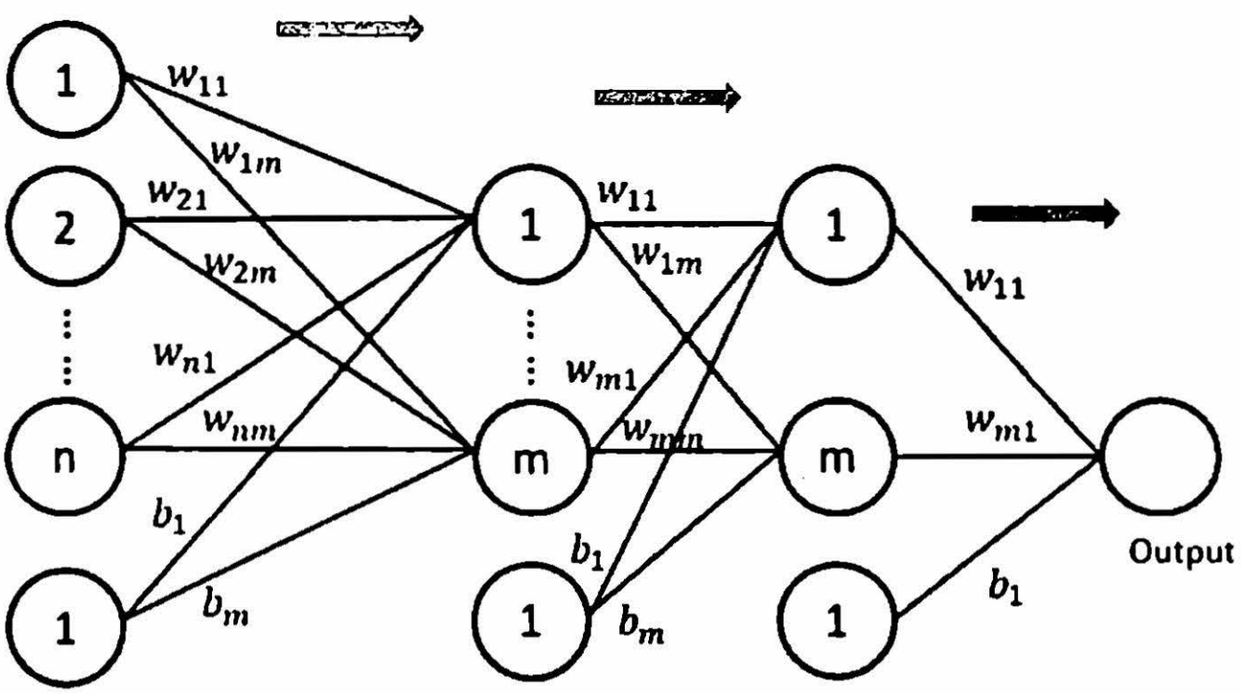
$$h = f(W^T X + b) \tag{3.11}$$

Cách biểu diễn mạng nơ-ron dưới dạng ma trận như đã trình bày rất hữu ích trong việc tính toán khi thực hiện các phép lan truyền xuôi và lan truyền ngược.

3.4.2 Cách biểu diễn lan truyền xuôi và lan truyền ngược

Lan truyền xuôi:

Lan truyền xuôi (forward propagation) là quá trình tính toán thông qua các tầng ẩn mà dữ liệu đầu vào được cho ra kết quả trong mô hình. Ta kí hiệu mỗi tầng i trong mạng được cấu tạo từ 3 phần bao gồm input $X^{(i)}$, ma trận trọng số $W^{(i)}$ và hàm kích hoạt $f^{(i)}$, trong đó, output của tầng này sẽ là input của tầng tiếp theo.



Hình 3.11: Quá trình lan truyền xuôi.

Như Hình 3.11 là kiến trúc mạng gồm 4 tầng (1 input layer, 2 hidden layer và 1 output layer).

$$
\begin{aligned}
X^{(1)} &= f^{(1)}(W^{(1)T} \times Input + b^{(1)}) \\
X^{(2)} &= f^{(2)}(W^{(2)T} \times X^{(1)} + b^{(2)}) \\
Output &= f^{(3)}(W^{(3)T} \times X^{(2)} + b^{(3)})
\end{aligned} \tag{3.12}
$$

Với cách tính (3.12) thì hoàn toàn có thể mở rộng một cách tổng quát với 1 input layer, n hidden layer và 1 output layer như sau:

$$
\begin{aligned}
X^{(1)} &= f^{(1)}(W^{(1)T} \times Input + b^{(1)}) \\
X^{(i)} &= f^{(i)}(W^{(i)T} \times X^{(i-1)} + b^{(i)}), i = \overline{2,n} \\
Output &= f^{(n)}(W^{(n)T} \times X^{(n)} + b^{(n+1)})
\end{aligned} \tag{3.13}
$$

Lan truyền ngược

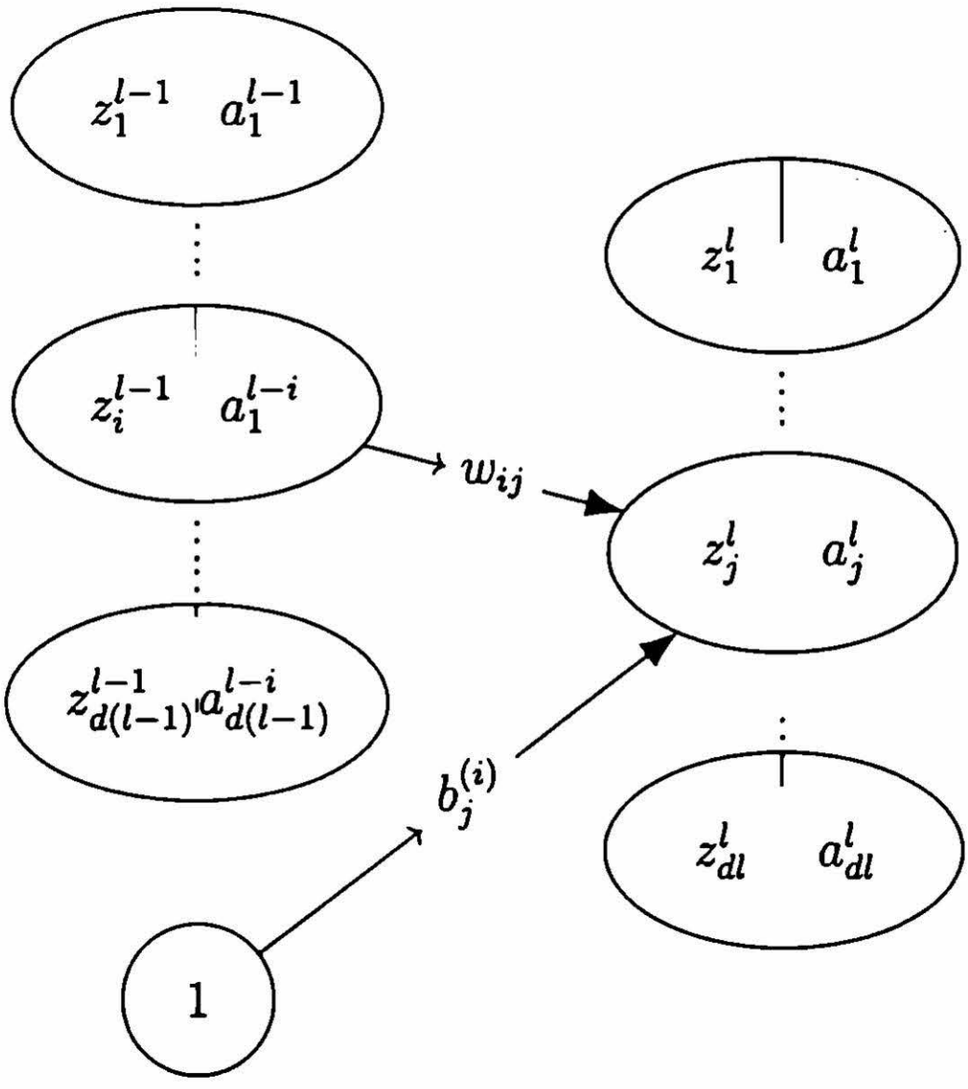
Lan truyền ngược (backward propagation hay back propagation) là quá trình cập nhật lại các trọng số để tìm ra bộ trọng số phù hợp sẽ được gọi là mô hình cho bài toán cần giải quyết. Khác với lan truyền xuôi khi sử dụng hàm kích hoạt, lan truyền ngược dựa trên việc tính đạo hàm để cho ra kết quả. Để cập nhật được trọng số thì bắt buộc cần phải có hàm mất mát J.

Nhằm dễ dàng hơn trong các bước tính toán tiếp theo nên nhắc lại những biểu thức sau:

$$z^{(i)} = W^{(i)T} \times X^{(i)} + b^{(i)} \tag{3.14}$$

$$a^{(i)} = f(z^{(i)}) \tag{3.15}$$

Các biểu thức (3.14) và (3.15) được biểu diễn như Hình 3.12. Nhằm đỡ rối khi nhìn nên chỉ vẽ tượng trưng các đường nối cần thiết, thực tế sẽ mỗi perceptron ở tầng trước phải nối hết với các perceptron tầng sau.



Hình 3.12: Quá trình lan truyền ngược giữa 2 tầng trong mạng nơ-ron

Tiếp theo, chúng ta sẽ có được gradient của các trọng số ở từng layer l như sau:

$$\frac{\partial J}{\partial w_{ij}^{(l)}} = \frac{\partial J}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}} = e_j^{(l)} \times a_i^{(l-1)} \tag{3.16}$$

với

$$\begin{aligned} e_j^{(l)} &= \frac{\partial J}{\partial z_j^{(l)}} = \frac{\partial J}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}} \\ &= \left(\sum_{k=1}^{d^{(l+1)}} \frac{\partial J}{\partial z_k^{(l+1)}} \cdot \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} \right) f'(z_j^{(l)}) \\ &= (w_j^{(l+1)} \times e^{(l+1)}) f'(z_j^{(l)}) \end{aligned} \tag{3.17}$$

Trong đó $e^{(l+1)} = [e_1^{(l+1)}, e_2^{(l+1)}, ..., e_{d(l+1)}^{(l+1)}]^T$ và $w_j^{(l+1)}$ là toàn bộ phần tử hàng thứ j của ma trận $W^{(l+1)}$. Nhờ xuất hiện trong công thức (3.17) thể hiện việc $a_j^{(l)}$ đóng góp vào việc tính các $z_k^{(l+1)}$.

Với cách làm tương tự, ta có:

$$\frac{\partial J}{\partial b_j^{(l)}} = e_j^{(l)} \tag{3.18}$$

Từ đây chúng ta có thể mở rộng ra cho cách biểu diễn bằng ma trận như các công thức sau:

$$\frac{\partial J}{\partial W^{(l)}} = a^{(l-1)} e^{(l)T}$$

$$e^{(l-1)} = W^{(l)T} e^{(l)} f'(z^{(l)}) \tag{3.19}$$

$$\frac{\partial J}{\partial b^{(l)}} = e^{(l)}$$

3.4.3 Vì sao lại biểu diễn bằng ma trận?

Để trả lời câu hỏi này, hãy thử hiện thực các tính toán trong mạng nơ-ron theo cách lập trình câu lệnh truyền thống sử dụng vòng lặp (imperative) mà không dùng đến ma trận với ngôn ngữ minh họa là Python.

```
J = 0
for i = 1 to m:
    z[i] = b
    for j in range(len(w)):
        z[i] += w[j]*x[i][j]
    a[i] = Sigmoid(z[i])
    J += -[y[i]*log(a[i]) + (1-y[i])*log(1-y[i])]
    dz[i] = a[i] - y[i]
    for j in range(len(w)):
```

```
10                dw[j] += x[i][j]*dz[i]
11        J = J / m
12        for j in range(len(w)):
13            w[j] = w[j] - alpha*dw[j]/m
```

Code Listing 3.1: Lập trình câu lệnh truyền thống để tính toán trong mạng nơ-ron

Tiếp theo, hãy thử hiện thực các phép tính toán bằng cách sử dụng ma trận và các phép tính dành cho ma trận với ngôn ngữ Python như sau:

```
1       import numpy as np
2
3       Z = np.dot(w.T,X) + b
4       A = Sigmoid(Z) #1
5       dZ = A - Y
6       dW = (1/m) * X * dZ.T
7       dB = (1/m) * np.sum(dZ)
8       W = W - alpha * dW
9       b = b - alpha * dB
```

Code Listing 3.2: Lập trình sử dụng ma trận để thực hiện tính toán trong mạng nơ-ron

Dễ dàng thấy rằng, về mặt biểu diễn thì cách tính toán trong Code Listing 3.2 gọn gàng hơn và dễ hiểu hơn đối với những người có nhiều kiến thức về toán học nhưng chưa có nhiều kiến thức về lập trình. Về mặt hiện thực, cách tính toán bằng ma trận trong Code Listing 3.2 cho phép thực hiện song song trên những máy tính có kiến trúc hỗ trợ đa luồng. Điều này cũng cho thấy rằng vì sao các kiến trúc mạng nơ-ron học sâu thì thường yêu cầu các perceptron trong cùng một tầng phải cùng sử dụng một hàm kích hoạt nhằm hỗ trợ việc tính toán song song.

3.4.4 Kỷ nguyên mới của mạng nơ-ron

Các chương vừa trình bày cho thấy rằng mạng nơ-ron có một tiềm năng tính toán rất lớn để giải quyết nhiều bài toán có tính thực tế cao. Tuy nhiên, có một thời gian dài mạng nơ-ron không được sử dụng hay còn gọi là chìm đắm trong "màn đêm".

Để giải thích việc này có thể quay lại việc lan truyền xuôi và lan truyền ngược sử dụng ma trận. Về mặt lý thuyết thì việc được lan truyền qua quá nhiều tầng sẽ khiến bộ trọng số bị suy biến và các tầng sau sẽ không còn học được nhiều nữa. Về mặt hiện thực, mạng nơ-ron nhiều tầng sử dụng các ma trận có kích thước rất lớn. Điều này đòi hỏi nhiều tài nguyên tính toán khiến cho những kiến trúc máy tính thế hệ trước không đủ khả năng xử lý và tính toán trên những ma trận lớn như vậy.

Hai vấn đề trên đã dần được giải quyết nhờ vào các bước tiến vượt bậc. Về mặt lý thuyết thì đã xuất hiện những hàm kích hoạt giúp giảm thiểu việc suy biến dữ liệu như đã giới thiệu trong Mục 3.3. Ngoài ra, về mặt phần cứng, sự xuất hiện của những mạng hỗ trợ đồ họa đã giúp cho việc tính toán trên ma trận có kích thước lớn rất nhanh và hiệu quả. Tiêu biểu là sự ra đời của GPU (Graphics Processing Unit).

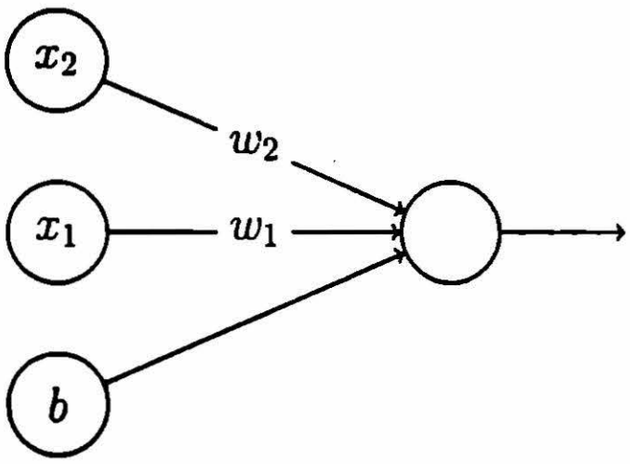
Như tên gọi của mình, GPU được dùng để hỗ trợ xử lý các dạng biểu diễn hình học trong đồ họa máy tính. Trong máy tính, các hình ảnh được biểu diễn bằng ma trận các *điểm ảnh* (pixel), vì vậy kiến trúc nội tại của GPU được xây dựng để hỗ trợ tối đa các phép tính toán trên ma trận để có thể xử lý hình ảnh tốt nhất. Thêm vào đó, GPU ngày nay lại được xử lý đa luồng và tiếp nhận thông tin gấp nhiều lần CPU để hỗ trợ đồ họa tạo hiệu ứng tối đa. Đây cũng là lý do mà chúng ta phải dùng ma trận hóa ở phần trước để tính toán khi hiện thực quá trình xử lý tính toán trên mạng nơ-ron và cũng là lý do vì sao các framework về học sâu ngày nay **chỉ cho dùng một hàm kích hoạt cho mỗi tầng.**

3.5 Kết chương

Qua chương này, chúng ta đã tìm hiểu và phân tích được cách thức hoạt động của mạng học sâu mà ngày nay được sử dụng ngoài thực tế. Ngoài ra, các vấn đề liên quan tới GPU và ma trận hóa cũng đã được trình bày để thấy được sự ảnh hưởng của phần cứng và sự hỗ trợ của ngôn ngữ khi chúng ta huấn luyện mạng học sâu. Từ đó chúng ta có phương pháp lựa chọn phần cứng, ngôn ngữ và thiết kế mô hình mạng hợp lý để tối ưu năng suất nhất có thể.

3.6 Bài tập

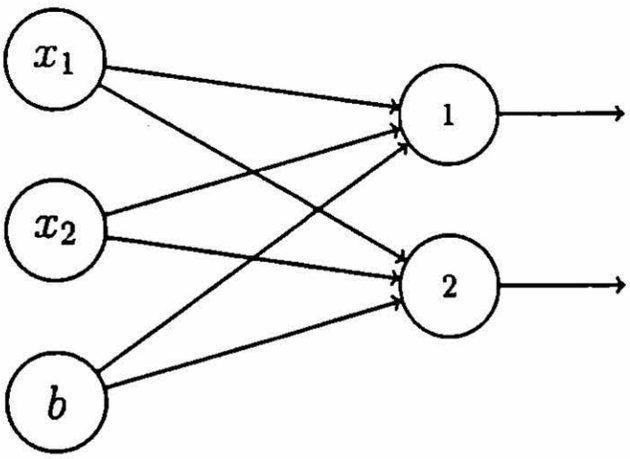
Bài tập 3.6.1. Cho mạng nơ-ron như hình vẽ sau:



Hình 3.13: Kiến trúc mạng nơ-ron của Bài tập 3.6.1

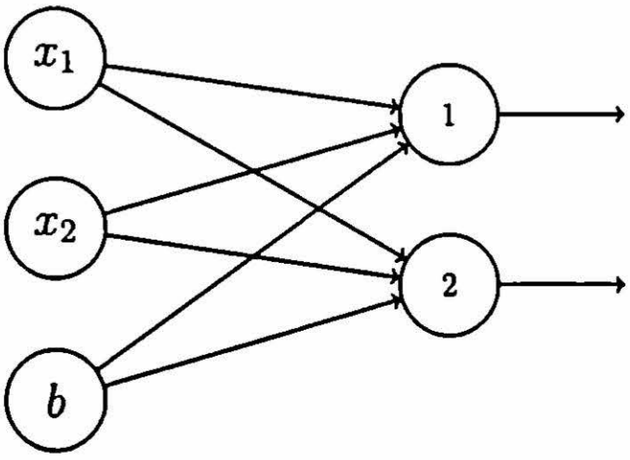
Biết rằng $w_1 = w_2 = w_{bias} = 0.25$, $x_1 = 0.1, x_2 = 0.2, bias = 1$ và output mong đợi là 1. Hãy tính output của mạng và các trọng số và bias mới(với hệ số học $\alpha = 0.1$ và hàm mất mát là *Cross Entropy*) của mạng khi hàm kích hoạt là hàm *sigmoid*.

Bài tập 3.6.2. Cho mạng nơ-ron như Hình 3.6.2. Biết rằng các trọng số và bias đều bằng 0.5, $x_1 = 0.1, x_2 = 0.2$ và output 1, 2 mong đợi lần lượt là 0, 1. Hãy dựa theo kiến thức của chương này, tìm ra các ma trận được tạo ra và dùng vectorize để tính toán thực hiện lan truyền xuôi.



Hình 3.14: Kiến trúc mạng nơ-ron của Bài tập 3.6.2

Bài tập 3.6.3. Cho mạng nơ-ron như hình vẽ sau:



Hình 3.15: Kiến trúc mạng nơ-ron của Bài tập 3.6.3

Biết rằng các trọng số và bias đều bằng 0.5, $x_1 = 0.1, x_2 = 0.2$ và output 1, 2 mong đợi lần lượt là 0, 1. Hãy tính output của mạng và các trọng số và bias mới (với hệ số học $\alpha = 0.1$ và hàm mất mát là *Cross Entropy*) của mạng với các kích hoạt sau:

1. Hàm kích hoạt của output 1, 2 lần lượt là hàm *tanh*, *sigmoid*.
2. Hàm kích hoạt output 1, 2 lần lượt là hàm *ReLU*, *sigmoid*.